A

PROJECT REPORT

ON

"DESIGN AND IMPLEMENTATION OF DMA CONTROLLER SOC WITH AXI-LITE INTERFACE AND ASYNCHRONOUS FIFO USING OPENROAD FLOW"

Submitted by

Uyyala Vishnu Vardhan Reddy
23001A0448

Guided by

Prof.V.Sumalatha

TO

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING
JNTUA COLLEGE OF ENGINEERING, ANANTHAPURAMU

JNTUA COLLEGE OF ENGINEERING, ANANTHAPURAMU

ACADEMIC YEAR: 2025-2026

INDEX

S. No.	Title	Page No.
1	Abstract	3
2	Introduction	4
3	System Requirements	5
3.1	Functional Requirements	5
3.2	Non-Functional Requirements	6
4	System Architecture	7
4.1	Block Diagram	7
4.2	Module Description	11
5	Design Specification	13
5.1	AXI-Lite Register Map	13
5.2	FIFO Design	16
5.3	DMA Engine	18
6	Implementation Details	21
6.1	RTL Design	21
6.2	Verification Methodology	24
6.3	Physical Design Flow Using OpenROAD	27
6.4	Flow Steps	29
7	Results	34
7.1	Simulation Results	34
7.2	Physical Design Results	37
8	Limitations	39
9	Future Work	41
10	Conclusion	44
11	References	44

1.Abstract

This project implements a simple System-on-Chip integrating a Direct Memory Access controller with an AXI4-Lite slave interface for configuration and control. The design enables programmed data movement without continuous processor involvement, improving system efficiency and reducing CPU overhead. To support clock domain crossing between control and data paths, an asynchronous FIFO is used, ensuring safe and reliable data transfer between independent clock domains.

The DMA engine manages transfer initiation, progress tracking, and completion signaling through an interrupt output. The complete RTL design is written in Verilog and verified using a self-checking testbench with waveform inspection in GTKWave. Synthesis is performed using Yosys, and the physical design is completed using the OpenROAD flow, covering floorplanning, placement, clock tree synthesis, routing, and GDS generation. The final layout is analyzed in KLayout.

This project demonstrates a full open-source RTL-to-GDS ASIC implementation flow and provides practical exposure to SoC integration, AXI-Lite based control, clock domain crossing using asynchronous FIFOs, and modern open-source VLSI design tools

2. Introduction

Modern embedded systems and System-on-Chip (SoC) architectures integrate multiple processing units, memory blocks, and peripheral interfaces on a single chip. In such systems, efficient data movement between memory and peripherals is critical for achieving high performance and optimal resource utilization.

Traditionally, data transfers are managed directly by the central processing unit (CPU), where the processor reads data from a source location and writes it to a destination using programmed I/O operations.

Although simple to implement, CPU-driven data transfer has several limitations. Each transfer operation consumes valuable processor cycles, increases instruction overhead, and blocks the CPU from executing other computational tasks. As data rates increase—particularly in applications such as multimedia processing, networking, storage systems, and high-speed sensor interfaces—the CPU becomes a performance bottleneck. Additionally, continuous CPU involvement in data movement increases memory bus contention and degrades overall system throughput.

To address these challenges, Direct Memory Access (DMA) controllers are widely used in modern SoCs. A DMA controller is a specialized hardware block that autonomously transfers data between memory and peripheral devices without continuous CPU intervention. The CPU initializes the transfer by programming configuration registers (such as source address, destination address, and transfer size), after which the DMA

engine independently performs the data movement. Upon completion, the DMA controller typically generates a status update or interrupt signal to notify the processor.

The use of DMA significantly improves system performance by:

- Reducing CPU overhead
- Improving bus bandwidth utilization
- Lowering transfer latency
- Enabling parallel execution of computation and data movement

In AMBA-based SoC architectures, the Advanced eXtensible Interface (AXI) protocol is widely adopted for high-performance communication between IP blocks. AXI4-Lite, a simplified subset of the AXI protocol, is commonly used for register-level control interfaces. It provides a lightweight, low-complexity mechanism for configuring hardware peripherals.

In this project, a small SoC-level DMA controller is designed and implemented with an AXI4-Lite slave interface for configuration and control. The AXI4-Lite interface allows a processor to program key parameters such as:

- Source memory address
- Destination memory address
- Transfer length
- Control and status registers

To support reliable operation in multi-clock systems, an asynchronous FIFO is integrated into the design. In practical SoCs, different subsystems often operate at independent clock frequencies. Direct signal transfer between clock domains can lead to metastability and data corruption. The asynchronous FIFO ensures safe clock domain crossing (CDC) using proper synchronization techniques, enabling robust data transfer between the control logic and the DMA engine.

The design is developed using a modular RTL architecture written in SystemVerilog. Each functional block—AXI-Lite slave interface, DMA read engine, DMA write engine, and asynchronous FIFO—is implemented as an independent module to promote scalability and reuse.

Furthermore, the complete design follows an ASIC implementation flow using open-source Electronic Design Automation (EDA) tools. RTL synthesis is performed using Yosys, while physical design—including floorplanning, placement, clock tree synthesis (CTS), and routing—is carried out using OpenROAD. The project demonstrates a complete RTL-to-GDSII methodology, providing hands-on exposure to industry-standard digital design and physical implementation processes.

By combining protocol-based SoC design, clock domain crossing techniques, and ASIC implementation flow, this project provides practical insight into real-world hardware system development and IP design methodologies.

3. System Requirements

This section defines the functional and non-functional requirements of the proposed SoC-level DMA controller. The requirements are derived from typical SoC data movement scenarios and the constraints of implementing the design using an ASIC flow with open-source tools. The system must provide programmable configuration, reliable data transfer control, safe clock domain crossing, and compatibility with synthesis and physical design tools.

The requirements are categorized into functional requirements and non-functional requirements.

3.1 Functional Requirements

3.1.1 AXI4-Lite Slave Interface for Configuration

The system shall implement an AXI4-Lite compliant slave interface to allow a processor or host controller to configure and monitor the DMA controller. The AXI4-Lite interface is used exclusively for control and status register access and does not carry the DMA data payload.

The interface must support:

- Register write transactions for configuring source address, destination address, transfer length, and control signals.
- Register read transactions for accessing status information such as completion flag and error status.
- Proper VALID/READY handshaking to ensure reliable and protocol-compliant communication.
- Single-beat transfers as defined by the AXI4-Lite specification.

The AXI-Lite interface must guarantee that configuration data is captured correctly and that no data corruption occurs during register programming.

3.1.2 Programmable Transfer Length

The DMA controller shall support a programmable transfer length register. This register defines the total number of data words to be transferred in a single DMA transaction.

The transfer length must:

- Be written through the AXI-Lite interface.
- Be latched internally when the start command is asserted.
- Be used to initialize an internal down-counter.
- Determine the completion condition when the counter reaches zero.

This programmability allows the system to handle varying transfer sizes without modifying hardware logic, increasing flexibility and reuse across multiple applications.

3.1.3 DMA Engine Data Handling Through FIFO

Once a transfer is initiated, the DMA engine shall autonomously control the data movement process without further CPU intervention.

The DMA engine must:

- Read data from the FIFO when valid data is available.
- Stall gracefully when the FIFO is empty.
- Continue operation until the programmed transfer length is exhausted.
- Maintain proper synchronization between FIFO status signals and internal state machine control.

The FIFO-based architecture decouples the data source from the DMA engine. This improves modularity and allows smooth operation even when data arrival timing is irregular or burst-based.

3.1.4 Interrupt on Completion

The system shall generate a dedicated interrupt signal upon completion of a DMA transfer.

The interrupt must:

- Assert only after the full programmed transfer length has been processed.
- Remain asserted until cleared by reset or by initiating a new transfer.
- Be synchronous to the control clock domain.

This interrupt mechanism allows the processor to detect transfer completion efficiently and perform post-transfer tasks such as buffer management or triggering subsequent operations.

3.1.5 Support for Two Independent Clock Domains

The system shall support operation across two independent clock domains to reflect realistic SoC integration scenarios.

- The AXI-Lite control interface and FIFO write logic shall operate in the AXI clock domain.
- The DMA engine and FIFO read logic shall operate in a separate DMA clock domain.

An asynchronous FIFO shall be implemented to safely transfer data between these clock domains. The FIFO must:

- Use proper pointer synchronization techniques (e.g., Gray-coded pointers).
- Prevent metastability through multi-stage flip-flop synchronizers.
- Maintain data integrity under asynchronous clock conditions.

This requirement ensures reliable clock domain crossing (CDC) and enables integration into multi-frequency SoC environments.

3.2 Non-Functional Requirements

3.2.1 Synthesis Compatibility

The RTL design shall be fully synthesizable using open-source synthesis tools such as Yosys. The design must:

- Avoid simulation-only constructs.
- Use synchronous design practices.
- Ensure no combinational loops exist.
- Follow standard RTL coding guidelines.

3.2.2 Physical Design Compatibility

The design shall be implementable using OpenROAD in a complete RTL-to-GDS flow. It must:

- Meet basic timing closure constraints.
- Avoid excessive fanout or long combinational paths.
- Support placement and routing without structural conflicts.

3.2.3 Modularity and Scalability

The architecture shall be modular, separating:

- AXI-Lite interface
- DMA control logic
- FIFO block
- Interrupt logic

This modularity enables future extensions such as:

- AXI4 full data interface
- Burst support
- Multi-channel DMA
- Scatter-gather support

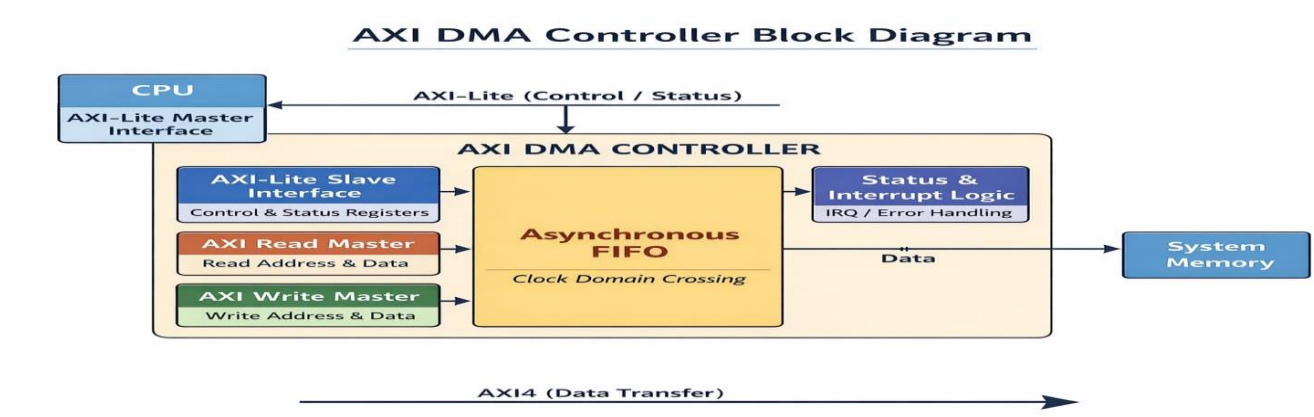
3.2.4 Robustness and Reliability

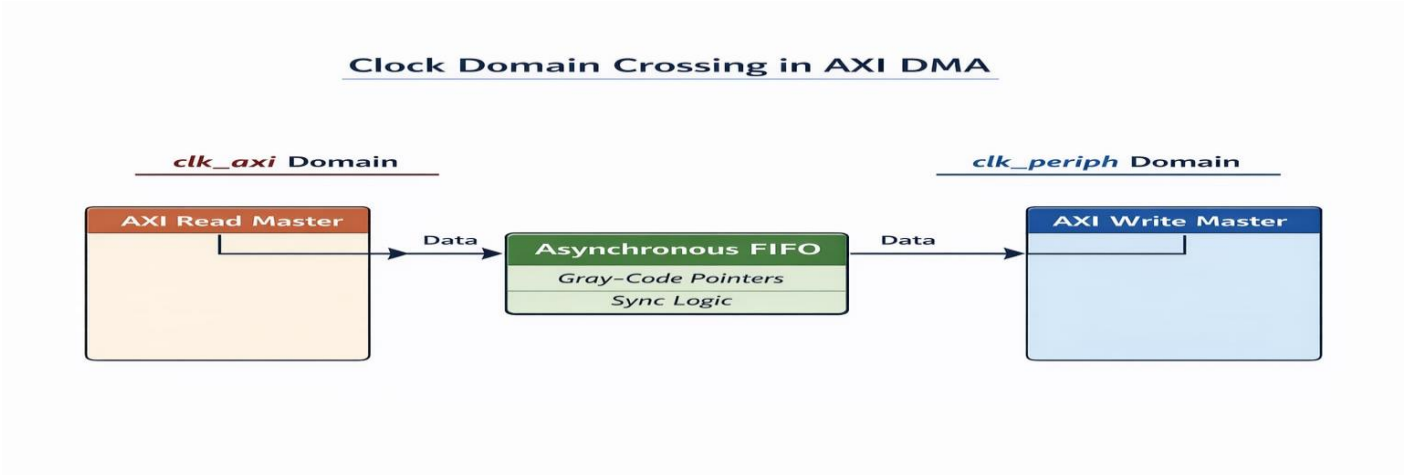
The design must:

- Handle FIFO empty and full conditions safely.
- Avoid race conditions across clock domains.
- Ensure deterministic completion signaling.
- Maintain consistent reset behavior across all modules.

4. System Architecture

4.1 Block Diagram





The proposed system is organized as a small SoC-level DMA subsystem controlled through an AXI4-Lite interface and operating across two independent clock domains. The architecture is modular and divided into four primary functional blocks:

1. AXI-Lite Register Block
2. Asynchronous FIFO (Clock Domain Crossing Unit)
3. DMA Engine
4. Interrupt Logic

This modular organization improves scalability, simplifies verification, and ensures safe clock domain crossing while remaining fully synthesizable for ASIC implementation.

3.2 AXI-Lite Register Block

The AXI-Lite Register Block serves as the configuration and status interface between the processor and the DMA subsystem. It is memory-mapped and accessible via the AXI4-Lite protocol.

This block performs the following functions:

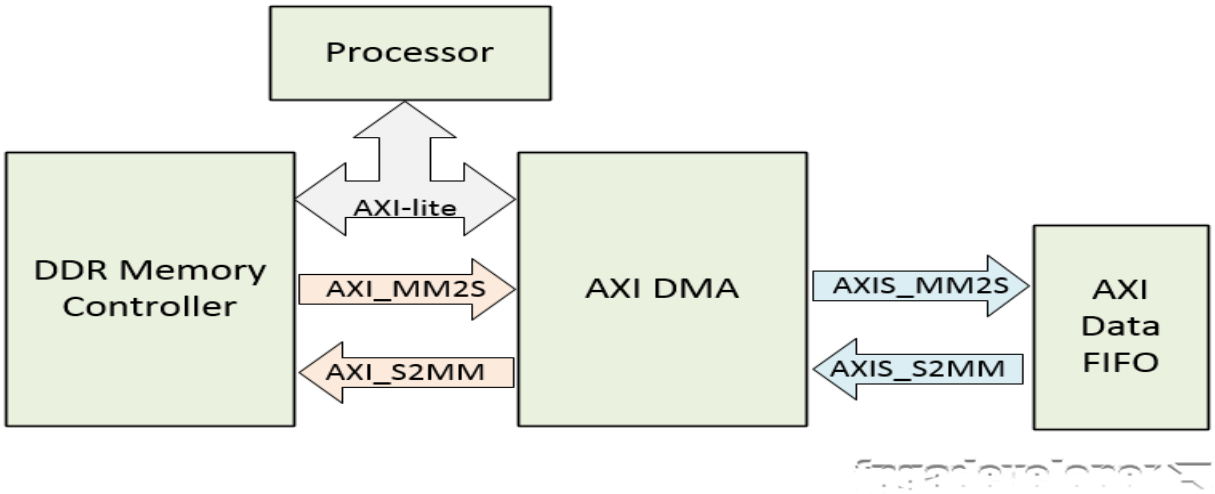
- Captures the programmed transfer length.
- Latches the start control bit.
- Provides status signals such as transfer completion.
- Generates control signals for the DMA engine.

Since AXI4-Lite is a lightweight subset of the AXI protocol, it supports single-beat read and write transactions with separate address and data channels. The block operates entirely in the AXI clock domain.

The control registers typically include:

- Transfer Length Register
- Control Register (Start bit)
- Status Register (Done flag)

This block ensures protocol-compliant handshaking using VALID and READY signals and guarantees reliable configuration without data corruption.



3.3 Asynchronous FIFO (Clock Domain Crossing)

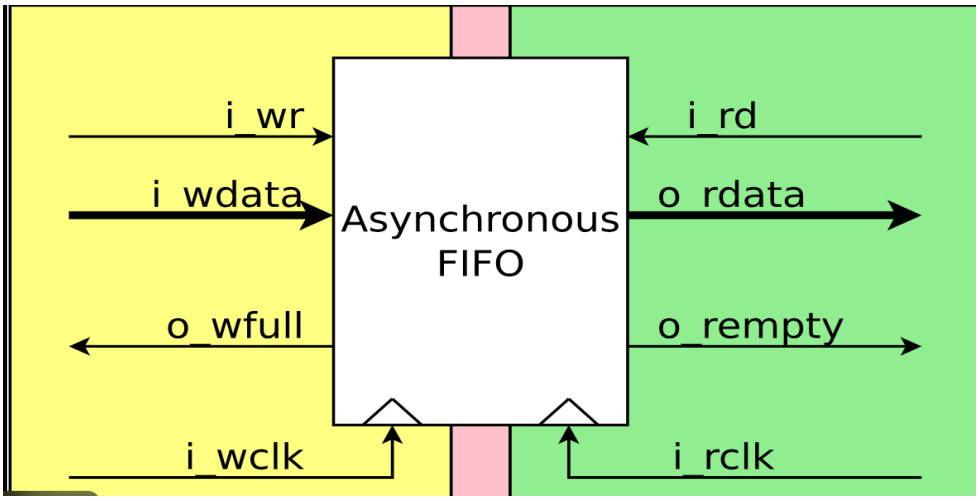
In practical SoC environments, different subsystems operate at different clock frequencies. To support this, the architecture includes an asynchronous FIFO to safely transfer data between the AXI clock domain and the DMA clock domain.

The FIFO performs the following functions:

- Buffers data generated in the AXI clock domain.
- Supplies data to the DMA engine in the DMA clock domain.
- Prevents overflow using a FULL flag.
- Prevents underflow using an EMPTY flag.

The FIFO internally uses pointer synchronization techniques to mitigate metastability issues during clock domain crossing. Typically, Gray-coded pointers and multi-stage synchronizers are used to ensure safe data transfer.

By isolating the two clock domains, the design achieves robust CDC handling and realistic SoC integration capability.



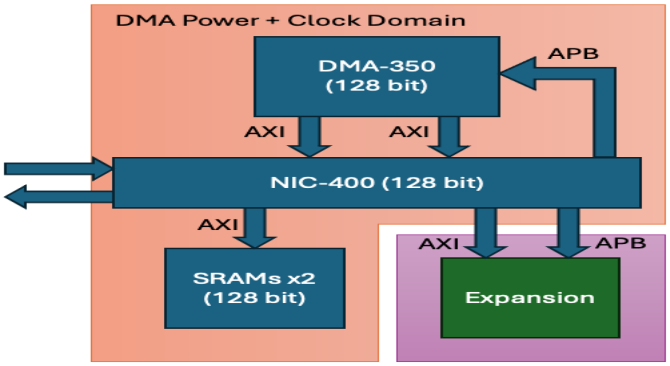
3.4 DMA Engine

The DMA Engine is the core functional block responsible for autonomous data transfer. It operates in the DMA clock domain and performs the following tasks:

- Monitors the start signal from the control block.
- Reads data from the FIFO when valid data is available.
- Maintains an internal counter initialized with the programmed transfer length.
- Decrements the counter after each successful data transfer.
- Detects transfer completion when the counter reaches zero.

The DMA engine is implemented using a finite state machine (FSM) to ensure deterministic and predictable operation. It stalls automatically if the FIFO is empty, ensuring no invalid data is processed.

This autonomous operation allows the CPU to perform other tasks while the DMA engine handles bulk data movement.



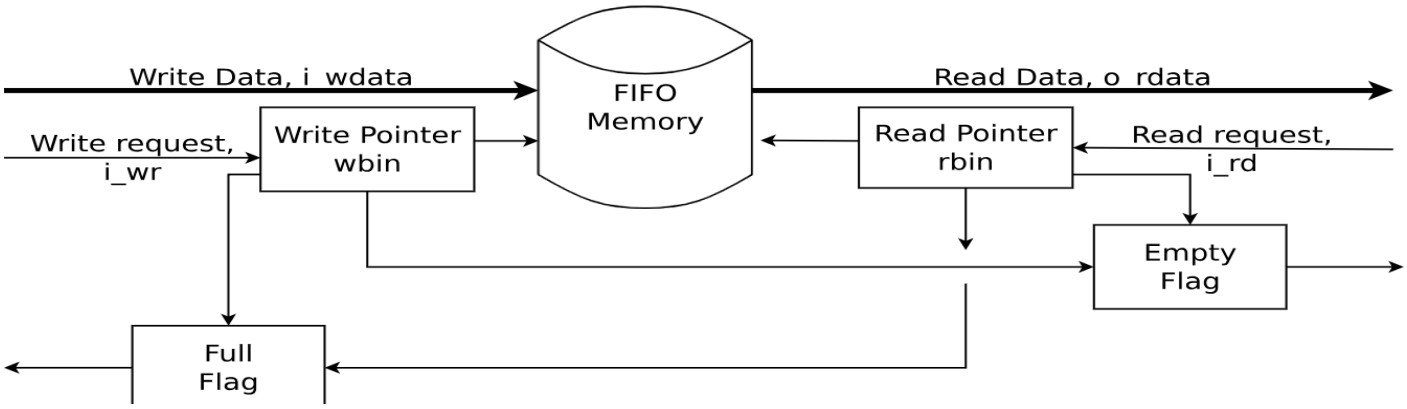
3.5 Interrupt Logic

The Interrupt Logic block generates a completion interrupt when the DMA transfer finishes.

Key characteristics:

- The interrupt asserts only after the full programmed transfer length has been processed.
- It remains asserted until cleared by reset or a new transfer initiation.
- It is synchronized to the AXI clock domain to ensure safe interaction with the processor.

This mechanism eliminates the need for continuous software polling and improves system efficiency.



3.6 Architectural Advantages

The proposed modular architecture provides several advantages:

- Clear separation between control path and data path.
- Safe clock domain crossing using asynchronous FIFO.
- Autonomous DMA operation after configuration.
- Simplified verification due to block-level isolation.
- Fully synthesizable structure compatible with ASIC flow.

The architecture is intentionally designed to be scalable. It can be extended in future work to support:

- Full AXI4 data interfaces
- Burst transfers
- Multi-channel DMA
- Scatter-gather capability
- Error detection and reporting

4.2 Module Description

This section describes the functionality and responsibilities of each module in the DMA SoC design. The system is structured in a modular fashion to enhance readability, simplify verification, and enable future scalability. Each module performs a clearly defined role within the overall architecture.

4.2.1 Dma_soc – Top-Level Integration Module

The **Dma_soc** module serves as the top-level integration unit of the DMA subsystem. It instantiates and interconnects all major submodules, including:

- AXI-Lite register block
- Asynchronous FIFO
- DMA engine
- Interrupt logic

This module provides the external interfaces of the system, including:

- AXI-Lite control interface ports
- AXI clock and DMA clock inputs

- Reset signal
- Interrupt output signal

The top-level module is responsible for routing control signals from the AXI-Lite domain to the DMA engine and managing data movement across clock domains via the asynchronous FIFO. It ensures proper synchronization and signal integrity between different functional blocks.

4.2.2 Axi_lite_regs – Control and Status Register Block

The **Axi_lite_regs** module implements the AXI4-Lite slave interface used to configure and monitor the DMA controller. It acts as the bridge between software (processor) and hardware logic.

This module provides programmable registers for:

- Source address
- Destination address
- Transfer length
- Start control bit

It also exposes status information such as:

- Transfer completion (Done flag)
- Interrupt status

The module operates entirely in the AXI clock domain. It follows the AXI4-Lite protocol, including proper VALID/READY handshaking and single-beat transaction support. Upon receiving a start command, the register values are latched and passed to the DMA engine for execution.

4.2.3 Async_fifo – Clock Domain Crossing Buffer

The **Async_fifo** module implements an asynchronous FIFO to safely transfer data between two independent clock domains.

- The write side operates in the AXI clock domain.
- The read side operates in the DMA clock domain.

Since the two clock domains may operate at different frequencies, direct signal transfer is unsafe and may cause metastability. The asynchronous FIFO prevents this issue by internally implementing pointer synchronization logic.

The FIFO provides:

- Full flag to prevent overflow
- Empty flag to prevent underflow
- Buffered data storage between producer and consumer

By decoupling data production and consumption rates, the FIFO enables stable operation even when clock frequencies differ significantly.

4.2.4 Dma_engine – DMA Control and Data Processing Logic

The **Dma_engine** module is the core functional unit of the DMA subsystem. It operates in the DMA clock domain and performs autonomous data transfer once initiated.

Its primary responsibilities include:

- Monitoring the start signal
- Reading data from the FIFO
- Tracking the programmed transfer length
- Maintaining an internal transfer counter
- Determining transfer completion

The DMA engine is typically implemented using a finite state machine (FSM) to ensure deterministic behavior. It decrements an internal counter for each data word transferred. When the counter reaches zero, the module asserts the done signal and triggers the interrupt logic.

This design allows the CPU to offload bulk data movement tasks and operate independently during transfer execution.

4.2.5 Tb_dma_soc – Verification Testbench

The **Tb_dma_soc** module is the simulation testbench used to verify the functional correctness of the DMA SoC design.

The testbench performs the following tasks:

- Generates AXI and DMA clock signals
- Applies reset sequences
- Programs AXI-Lite control registers
- Initiates DMA transfers
- Monitors the interrupt and completion signals
- Dumps waveform files for analysis using GTKWave

The testbench is used exclusively for simulation and validation. It is not part of the synthesizable hardware and is excluded from the ASIC implementation flow.

5. Design Specification

5.1 AXI-Lite Register Map

The DMA controller exposes a set of memory-mapped control registers through an AXI4-Lite slave interface. These registers allow the processor or verification testbench to configure, initiate, and monitor DMA transfers.

All registers:

- Are 32 bits wide
- Are accessed using aligned word transactions

- Are located at fixed address offsets from the DMA base address
- Operate in the AXI clock domain

The AXI-Lite interface supports single-beat read and write transactions with proper VALID/READY handshaking as defined by the AMBA AXI4-Lite protocol.

5.1.1 Register Summary

Address Offset	Register Name	Access Type	Description
0x00	START	Write	Initiates DMA transfer
0x04	LENGTH	Read/Write	Specifies transfer length
0x08	STATUS	Read	Indicates completion status

5.1.2 START Register (0x00)

Address Offset: `0x00`
Access Type: Write-only
Width: 32 bits

Description

The START register is used to initiate a DMA transfer.

- Writing a logic '1' to bit[0] triggers the DMA engine to begin processing.
- The DMA engine latches the start command internally.
- Writing '0' clears the start signal.
- Bits [31:1] are reserved and must be written as zero.

Bit Field Definition

Bit	Name	Description
0	START	1 = Start DMA transfer
31:1	Reserved	Must be written as 0

The start command is typically self-clearing once latched by the DMA engine to prevent unintended repeated transfers.

5.1.3 LENGTH Register (0x04)

Address Offset: `0x04`
Access Type: Read/Write
Width: 32 bits

Description

The LENGTH register specifies the number of data words to be transferred during the DMA operation.

- The value written to this register is loaded into an internal transfer counter when the START command is asserted.

- The DMA engine decrements the counter after each successful data transfer.
- When the counter reaches zero, the transfer is considered complete.
- Upon completion, the interrupt signal is asserted.

Only positive, non-zero values are valid for initiating a transfer. Writing a value of zero results in no operation.

Bit Field Definition

Bits	Name	Description
31:0	LENGTH	Number of data words to transfer

5.1.4 STATUS Register (0x08)

Address Offset: `0x08`
Access Type: Read-only
Width: 32 bits

Description

The STATUS register provides information about the current state of the DMA controller.

Bit	Name	Description
0	DONE	1 = Transfer complete
31:1	Reserved	Always reads as 0

The DONE bit is set when the programmed transfer length has been fully processed. It is cleared upon reset or when a new START command is issued.

5.1.5 Operational Sequence

A typical DMA transfer sequence is as follows:

1. Processor writes transfer size to LENGTH register.
2. Processor writes ‘1’ to START register.
3. DMA engine begins reading data from FIFO.
4. Internal counter decrements on each transfer.
5. When counter reaches zero:
    - DONE bit is set.
    - Interrupt signal is asserted.
6. Processor reads STATUS register to confirm completion.

Design Considerations

- All register accesses follow AXI-Lite handshake timing.
- Register writes are synchronized to the AXI clock.
- The START signal is transferred safely to the DMA clock domain.
- Reserved bits ensure future extensibility.

5.2 FIFO Design

5.2.1 Overview

An asynchronous FIFO is required in this design to safely transfer data between two independent clock domains. The AXI-Lite control logic and data generation operate in the AXI clock domain, while the DMA engine processes data in the DMA clock domain.

Since these two clocks are asynchronous and may operate at different frequencies, direct signal transfer between domains can cause metastability, data corruption, and unpredictable behavior. The asynchronous FIFO provides a reliable buffering mechanism that decouples the producer (AXI domain) from the consumer (DMA domain), ensuring correct and stable data transfer across clock boundaries.

5.2.2 Need for Clock Domain Crossing (CDC)

In digital systems, metastability occurs when a signal changes near the clock sampling edge of another domain. Without proper synchronization, this can lead to:

- Incorrect data sampling
- Glitches
- Unpredictable system behavior
- Functional failure

To eliminate these risks, the design uses a dual-clock FIFO that internally manages clock domain crossing.

5.2.3 FIFO Architecture

The asynchronous FIFO uses:

- Separate write clock (AXI clock domain)
- Separate read clock (DMA clock domain)
- Independent write and read pointers
- Full and empty status flags

Write Side (AXI Clock Domain)

- Operates using the AXI clock.
- Accepts data when the FIFO is not full.
- Increments the write pointer on successful write.
- Generates the FULL flag when the FIFO reaches maximum capacity.

Read Side (DMA Clock Domain)

- Operates using the DMA clock.
- Supplies data to the DMA engine when FIFO is not empty.
- Increments the read pointer on successful read.
- Generates the EMPTY flag when no valid data is available.

This separation allows both domains to operate independently without direct clock synchronization.

5.2.4 Full and Empty Flag Generation

To prevent overflow and underflow conditions, the FIFO implements status flags:

Full Flag

Indicates that:

- The write pointer has reached the read pointer minus one.
- No additional data can be written until space becomes available.

Empty Flag

Indicates that:

- The read pointer equals the write pointer.
- No valid data is available for reading.

These flags are synchronized across clock domains using multi-stage flip-flop synchronizers to prevent metastability.

5.2.5 Pointer Synchronization Technique

To safely compare pointers across clock domains, the FIFO typically uses:

- Binary counters for local pointer increment
- Gray-coded pointers for cross-domain transfer
- Two-stage flip-flop synchronizers

Why Gray Code?

In binary counters, multiple bits may change simultaneously. If sampled asynchronously, this may cause incorrect pointer interpretation.

Gray code ensures that:

- Only one bit changes at a time.
- Pointer synchronization is safer.
- Metastability risk is minimized.

This technique is widely used in industrial asynchronous FIFO implementations.

5.2.6 FIFO Depth Selection

The FIFO depth is fixed at design time. The selected depth must balance:

- Throughput requirements
- Hardware resource utilization
- Clock frequency differences
- Latency tolerance

A larger FIFO depth:

- Provides better tolerance for clock frequency mismatch.
- Reduces risk of stalling.

However, it increases:

- Area usage
- Power consumption

The chosen depth ensures smooth data flow even when the producer and consumer operate at significantly different rates.

5.2.7 Design Advantages

The asynchronous FIFO provides:

- Safe clock domain crossing
- Independent operation of producer and consumer
- Protection against overflow and underflow
- Improved system robustness
- Better scalability for real SoC integration

5.2.8 Implementation Constraints

For ASIC implementation using open-source tools:

- The FIFO must be fully synthesizable.
- No simulation-only constructs are allowed.
- Pointer logic must be synchronous.
- Synchronizers must use standard flip-flop chains.

Proper timing constraints must be applied during synthesis and physical design to handle multi-clock paths correctly.

5.3 DMA Engine Design

5.3.1 Overview

The DMA Engine is the core functional block of the DMA subsystem. It controls the data transfer operation and manages the overall execution of a DMA transaction. The DMA engine operates entirely in the DMA clock domain and is triggered by a start command issued through the AXI-Lite control interface.

The DMA engine is implemented as a deterministic finite state machine (FSM). This ensures predictable behavior, clear transfer sequencing, and simplified verification.

5.3.2 Functional Responsibilities

The DMA engine performs the following key functions:

- Receives and synchronizes the start signal from the AXI clock domain.
- Loads the programmed transfer length into an internal counter.
- Reads data from the asynchronous FIFO.
- Tracks transfer progress using a decrementing counter.
- Detects transfer completion.
- Generates a done signal and interrupt (IRQ).
- Returns to idle state after completion.

5.3.3 Finite State Machine (FSM) Description

The DMA engine operates using two primary states:

1. Idle State

In the Idle state:

- The DMA engine waits for:
    - A valid start signal.
    - A non-zero transfer length.
- No FIFO read operations occur.
- The done and interrupt signals remain deasserted.
- The internal transfer counter is not active.

This state represents the inactive condition of the DMA controller.

Transition Condition:

- When the start signal is asserted and the programmed transfer length is non-zero, the engine transitions to the Active state.
- The transfer length value is loaded into the internal counter.

2. Active State

In the Active state:

- The DMA engine begins processing the transfer.
- It monitors the FIFO empty flag.
- FIFO read enable is asserted only when:
    - The FIFO is not empty.
    - The transfer counter is greater than zero.

This prevents FIFO underflow and ensures valid data transfer.

The engine remains in this state until the programmed transfer length has been fully processed.

5.3.4 FIFO Read Operation

During operation:

- The DMA engine asserts FIFO read enable when valid data is available.

- Each successful FIFO read corresponds to one data word transferred.
- The read operation is fully synchronized to the DMA clock domain.

The design ensures that:

- No read is attempted when FIFO is empty.
- No invalid data is processed.
- No clock-domain hazards occur.

5.3.5 Transfer Counter Logic

The DMA engine maintains an internal down-counter to track the number of remaining transfers.

Operation of the counter:

1. The counter is loaded with the programmed transfer length at the start of the operation.
2. The counter decrements on every successful FIFO read.
3. The counter value represents the number of remaining data words to transfer.

This mechanism provides precise control over the transfer length and ensures deterministic completion.

5.3.6 Transfer Completion

When the internal counter reaches zero:

- The DMA engine asserts the done signal.
- The interrupt (IRQ) signal is generated.
- No further FIFO reads occur.
- The FSM transitions back to the Idle state.

The interrupt informs the processor that the transfer has completed successfully. The interrupt remains asserted until cleared by reset or by issuing a new start command.

5.3.7 Design Characteristics

The DMA engine design provides:

- Deterministic state transitions
- Controlled FIFO access
- Safe operation in DMA clock domain
- Accurate transfer tracking
- Clean interrupt generation

5.3.8 Design Safety Considerations

To ensure robust operation:

- The start signal must be synchronized from AXI clock to DMA clock domain.
- The counter must be reset during system reset.
- FIFO empty condition must be checked before read enable assertion.

- All control signals must be synchronous to the DMA clock.

These precautions guarantee reliable operation during ASIC synthesis and physical implementation.

6. Implementation Details

6.1 RTL Design

The complete DMA SoC subsystem is implemented using synthesizable Verilog HDL following ASIC-compatible coding guidelines. Each functional block is written as an independent RTL module with clearly defined input and output interfaces. This modular structure improves readability, simplifies debugging, and enables scalable integration into larger SoC environments.

The top-level module (`dma_soc`) integrates the following submodules:

- AXI-Lite Register Block (`axi_lite_regs`)
- Asynchronous FIFO (`async_fifo`)
- DMA Engine (`dma_engine`)

All modules are written using synchronous design methodology with positive-edge-triggered flip-flops. Blocking assignments are avoided in sequential logic to prevent race conditions. Only non-blocking assignments (<=) are used in clocked always blocks to ensure correct simulation and synthesis behavior.

Clocking Architecture

The design operates across **two independent clock domains**:

1. **AXI Clock Domain (`axi_clk`)**
    - AXI-Lite slave interface
    - Control register logic
    - FIFO write logic
    - FIFO data generator (verification support)
2. **DMA Clock Domain (`dma_clk`)**
    - FIFO read logic
    - DMA engine state machine
    - Transfer counter
    - Interrupt generation logic

Since these clocks are asynchronous and may operate at different frequencies, direct signal sharing is avoided. Instead, the asynchronous FIFO serves as the clock domain crossing (CDC) boundary.

Reset Strategy

A single active-low asynchronous reset (`rst_n`) is used across the entire design.

- All state machines are reset to a known idle state.
- All counters are cleared.
- FIFO pointers are initialized.
- Interrupt and status signals are deasserted.

Although the reset signal is asynchronous, it is internally handled carefully in each clock domain to avoid metastability. During synthesis and physical design, reset trees can be optimized for proper distribution.

AXI-Lite Interface Implementation

The AXI-Lite interface follows a simplified but compliant handshake protocol:

- Independent read and write address channels
- Valid-ready handshake mechanism
- Single-beat transactions (as required by AXI4-Lite)

The interface handles:

- Write address and write data acceptance
- Write response generation
- Read address acceptance
- Read data return

All register writes occur only when both `AWVALID` and `WVALID` are asserted and the module responds with `AWREADY` and `WREADY`. Similarly, read transactions complete only after `ARVALID` and `RREADY` handshake successfully.

This ensures:

- No data corruption
- Proper backpressure handling
- Deterministic register programming

FIFO Data Generator (Verification Support)

To facilitate functional verification without requiring a complete external memory model, a simple FIFO data generator is implemented within the AXI clock domain.

Key characteristics:

- Generates sequential incrementing data pattern
- Writes data into FIFO when:
    - DMA transfer is active
    - FIFO is not full
- Stops when transfer completes

This pattern enables:

- Easy waveform validation
- Simple debugging in GTKWave
- Clear observation of FIFO write/read operations
- Verification of correct transfer length handling

This logic is synthesizable but intended mainly for simulation-based validation.

DMA Engine FSM Implementation

The DMA engine is implemented as a finite state machine (FSM) with clearly defined states:

1. **IDLE**
    - Waits for start command
    - Loads transfer length
    - Clears done flag
2. **ACTIVE**
    - Monitors FIFO empty flag
    - Issues read enable when data is available
    - Decrements internal counter
3. **COMPLETE**
    - Asserts `done`
    - Generates interrupt
    - Returns to IDLE

The FSM ensures deterministic behavior and avoids combinational feedback loops.

Synthesizability Considerations

The RTL is written to be fully compatible with:

- Yosys synthesis
- OpenROAD physical design flow

Key synthesis-friendly practices used:

- No `#delays`
- No unsynthesizable constructs
- No initial blocks in synthesizable modules
- Proper sensitivity lists
- Parameterizable FIFO depth
- Fixed data width (32-bit aligned)

The design is therefore ready for:

RTL Simulation → Gate-Level Netlist → Placement → Routing → GDS Generation

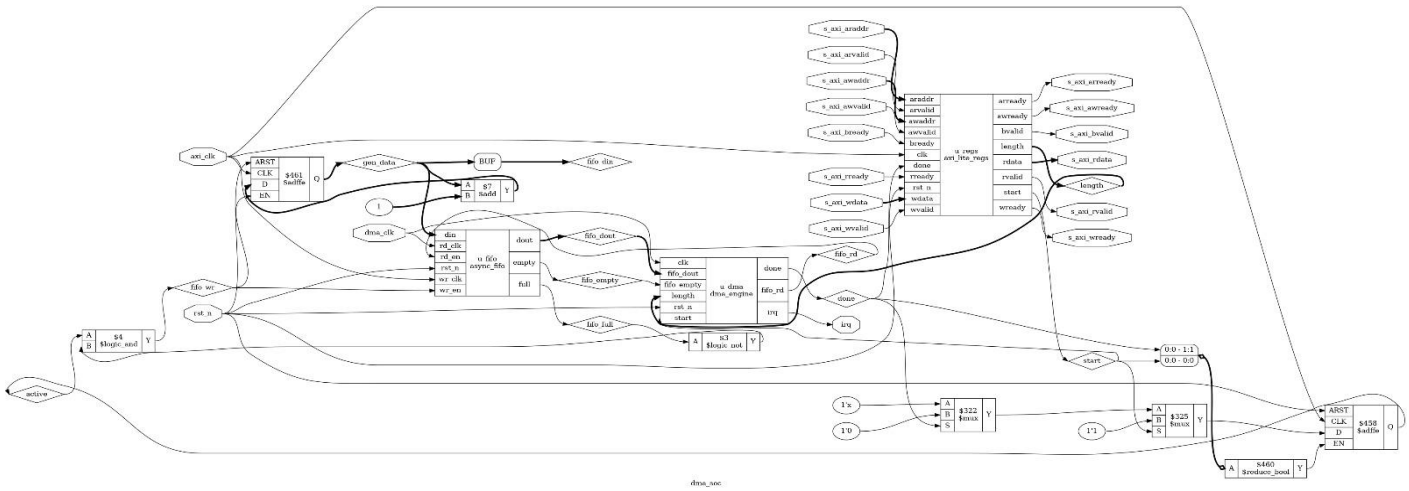


Figure 5.1: RTL Schematic of DMA SoC (Generated after synthesis)

6.2 Verification Methodology

Functional verification of the DMA SoC subsystem is performed using event-driven RTL simulation prior to synthesis. Simulation ensures that the logical behavior of the design is correct before proceeding to gate-level synthesis and physical design. The verification environment is built using Icarus Verilog as the simulator and GTKWave for waveform analysis.

Simulation Environment

The complete RTL design, including:

- `dma_soc` (top-level module)
- `axi_lite_regs`
- `async_fifo`
- `dma_engine`

is compiled together with the verification testbench `tb_dma_soc.sv`.

Simulation is executed using Icarus Verilog, which performs:

1. Compilation of RTL and testbench
2. Elaboration of module hierarchy
3. Event-driven simulation of signal transitions
4. Generation of a Value Change Dump (VCD) file

The VCD file captures all relevant signal activity for post-simulation waveform inspection.

Testbench Architecture

The testbench is designed to emulate a processor interacting with the DMA subsystem through the AXI-Lite interface.

It performs the following tasks:

Clock Generation

- Generates `axi_clk` for the control domain
- Generates `dma_clk` for the DMA domain
- Uses different clock frequencies to validate asynchronous FIFO behavior

Reset Generation

- Applies active-low reset (`rst_n`)
- Ensures all modules initialize correctly
- Releases reset after a defined simulation time

AXI-Lite Register Programming

The testbench performs AXI-Lite write transactions to:

- Program the transfer length register
- Assert the START register

It uses proper valid-ready handshake sequences to mimic real AXI-Lite behavior.

DMA Execution Monitoring

After initiating the transfer, the testbench:

- Waits for interrupt assertion
- Confirms transfer completion
- Verifies that no spurious interrupts occur

Waveform Analysis Using GTKWave

The simulation generates a `.vcd` file which is analyzed in GTKWave.

Waveform inspection provides detailed visibility into:

- AXI handshake timing
- FIFO write and read behavior
- DMA engine state transitions
- Transfer counter operation
- Interrupt generation

This graphical analysis is critical for validating correct temporal behavior and identifying subtle logic issues.

Functional Checks Performed

The following key behaviors are verified:

1. AXI-Lite Handshake Validation

- `AWVALID` and `WVALID` must be acknowledged by `AWREADY` and `WREADY`
- `ARVALID` must be acknowledged by `ARREADY`
- `RVALID` and `RREADY` handshake must complete successfully
- No register update should occur without proper handshake

2. FIFO Write Validation

- `wr_en` asserted only when FIFO is not full
- Data written matches expected incrementing pattern
- Full flag correctly prevents overflow

3. FIFO Read Validation

- `rd_en` asserted only when FIFO is not empty
- Read operations synchronized to `dma_clk`
- Empty flag prevents underflow

4. Transfer Counter Validation

- Counter loads programmed LENGTH value
- Counter decrements only on successful FIFO read
- Counter reaches zero exactly after programmed number of transfers
- No extra or missing transfers occur

5. Interrupt Generation Check

- `irq` asserted only after counter reaches zero
- No premature interrupt assertion
- Interrupt remains stable until reset or new transfer
- No glitching observed across clock domains

Cross-Domain Validation

Because the system uses two independent clocks, special attention is given to:

- Safe clock domain crossing through asynchronous FIFO
- No metastability propagation
- Correct pointer synchronization
- No data corruption across domain boundaries

Simulation uses different clock frequencies to stress-test CDC behavior.

Verification Coverage

Although UVM is not used in this project, the following coverage aspects are validated:

- Single transfer operation

- Multiple transfer operations (back-to-back)
- Small transfer lengths
- Large transfer lengths
- FIFO boundary conditions (almost full / almost empty)

This ensures confidence in functional correctness prior to synthesis.

6.3 Physical Design Flow Using OpenROAD

6.1 Toolchain

The physical implementation of the DMA SoC follows a complete RTL-to-GDSII flow using fully open-source Electronic Design Automation (EDA) tools. This flow transforms the synthesizable Verilog RTL into a manufacturable physical layout. Each tool in the flow performs a well-defined task, forming a cohesive ASIC implementation pipeline.

The overall flow can be summarized as:

RTL → Synthesis → Floorplanning → Placement → Clock Tree Synthesis → Routing → DRC → GDS

Yosys – RTL Synthesis

Yosys is used to synthesize the Verilog RTL into a gate-level netlist mapped to a target standard cell library.

During synthesis, Yosys performs:

- RTL elaboration
- Hierarchy resolution
- Logic optimization
- Removal of redundant logic
- Technology mapping to standard cells
- Netlist generation in Verilog format

The output of this stage is a gate-level netlist compatible with the selected standard cell library (e.g., Sky130).

Synthesis ensures:

- Structural correctness of the design
- No unresolved signals or floating nets
- No combinational loops
- Proper mapping of flip-flops and logic gates
- Hardware realizability

Additionally, synthesis reports provide:

- Cell count
- Area estimation
- Basic timing information
- Resource utilization

This stage confirms that the RTL is suitable for physical implementation.

OpenROAD – Physical Design (Place and Route)

OpenROAD performs the complete digital backend physical design process.

It takes the synthesized gate-level netlist and performs:

1. Floorplanning
    - Defines chip/core area
    - Places I/O pins
    - Sets placement boundaries
2. Global Placement
    - Places standard cells within the core
    - Optimizes for wirelength and congestion
3. Detailed Placement
    - Legalizes cell placement
    - Aligns cells to placement grid
4. Clock Tree Synthesis (CTS)
    - Inserts clock buffers
    - Balances clock skew
    - Reduces clock latency
5. Global Routing
    - Plans routing paths
6. Detailed Routing
    - Connects all nets physically
    - Follows metal layer constraints
7. Timing Optimization
    - Fixes setup/hold violations
    - Buffer insertion
    - Gate resizing

The final output of OpenROAD is:

- Routed DEF file
- Timing reports
- Power estimation reports
- Final GDSII layout file

This stage converts the logical netlist into a physical silicon layout ready for fabrication.

KLayout – Layout Visualization

KLayout is used to open and visually inspect the final GDSII layout generated by OpenROAD.

It allows:

- Viewing placement of standard cells
- Inspecting routing layers

- Zooming into metal interconnects
- Checking chip boundaries
- Visual confirmation of floorplan

Although it does not modify the layout, it provides important visual validation of the physical design.

Magic – Design Rule Checking (DRC)

Magic is used to perform basic Design Rule Checking (DRC) on the generated layout.

DRC ensures that:

- Minimum metal spacing rules are satisfied
- Width constraints are met
- No geometry overlaps violate technology rules
- Layout adheres to fabrication constraints

This step helps identify potential manufacturing violations before tape-out.

GTKWave – Waveform Visualization (Pre-Synthesis Verification)

GTKWave is used during the functional verification phase to analyze simulation waveforms.

Although not part of the physical design flow, it is critical for:

- Validating functional correctness
- Debugging control logic
- Verifying clock domain crossing
- Ensuring correct interrupt behavior

Only after functional correctness is verified through simulation does the design proceed to synthesis and physical implementation.

6.4 Flow Steps

The RTL-to-GDS implementation of the DMA SoC follows a structured ASIC physical design flow using open-source EDA tools. Each stage transforms the design from a behavioral description into a physically manufacturable layout while maintaining functional correctness and meeting physical design constraints such as timing, area, and routability.

The major stages of the flow are described below.

1. RTL Synthesis Using Yosys

The design process begins with synthesis of the Verilog RTL using Yosys.

Objectives of Synthesis:

- Convert behavioral RTL into gate-level netlist

- Perform logic optimization
- Map generic logic to standard cell library
- Eliminate redundant logic
- Generate timing-aware netlist

During synthesis, Yosys performs:

- Parsing and elaboration of RTL hierarchy
- Constant propagation and optimization
- FSM extraction and optimization
- Technology mapping to target library cells
- Removal of unused logic

The output is a **gate-level Verilog netlist** compatible with the selected standard cell library (e.g., Sky130). This netlist serves as the input to the backend physical design flow.

Synthesis reports provide:

- Total cell count
- Area estimation
- Logic depth
- Register count

This stage ensures that the design is structurally correct and physically realizable.

2. Floorplanning

Floorplanning defines the physical layout boundaries of the chip.

Key Floorplanning Tasks:

- Define die area dimensions
- Define core area dimensions
- Set core utilization percentage
- Place I/O pins
- Define power and ground rails
- Create placement blockages if required

Proper floorplanning ensures:

- Sufficient area for standard cells
- Adequate routing resources
- Reduced congestion
- Balanced aspect ratio

For small designs like this DMA SoC, a simple rectangular core with moderate utilization (50–70%) is typically sufficient.

3. Placement

Placement determines the exact location of all standard cells within the core region.

Objectives of Placement:

- Minimize total wire length
- Reduce routing congestion
- Improve timing performance
- Legalize cell positions to placement grid

Placement occurs in two phases:

Global Placement

- Rough positioning of cells
- Optimization for wire length and congestion

Detailed Placement

- Legalization to remove overlaps
- Alignment to standard cell rows
- Minor local optimizations

Good placement improves routing quality and reduces timing violations in later stages.

4. Clock Tree Synthesis (CTS)

Clock Tree Synthesis distributes the clock signal to all flip-flops and sequential elements.

Purpose of CTS:

- Minimize clock skew
- Control insertion delay
- Balance clock tree paths
- Improve timing reliability

During CTS:

- Clock buffers are inserted
- Clock routing topology is generated
- Skew between clock endpoints is minimized

Since the DMA SoC contains two clock domains (AXI and DMA), each domain receives its own clock tree.

Proper CTS is critical to:

- Avoid setup/hold timing violations
- Prevent metastability issues
- Ensure synchronous operation

5. Routing

Routing physically connects all placed cells according to the netlist.

Routing occurs in two stages:

Global Routing

- Determines approximate routing paths
- Assigns routing layers
- Estimates congestion

Detailed Routing

- Generates exact metal connections
- Follows metal width and spacing rules
- Avoids design rule violations
- Connects all nets completely

Proper routing ensures:

- No open circuits
- No short circuits
- Compliance with fabrication rules
- Acceptable signal integrity

6. Timing Optimization

After placement and routing, static timing analysis (STA) is performed.

If violations occur:

- Buffers are inserted
- Gates are resized
- Paths are optimized

The goal is to meet timing constraints such as setup and hold requirements.

For a small DMA design, timing closure is typically straightforward.

7. GDS Generation

After successful routing and timing closure, the final layout is exported in **GDSII format**.

The GDS file contains:

- Standard cell geometry
- Routing metal layers
- Via connections
- Power grid

- I/O pin definitions

This file represents the manufacturable layout of the chip and is the final output of the physical design flow.

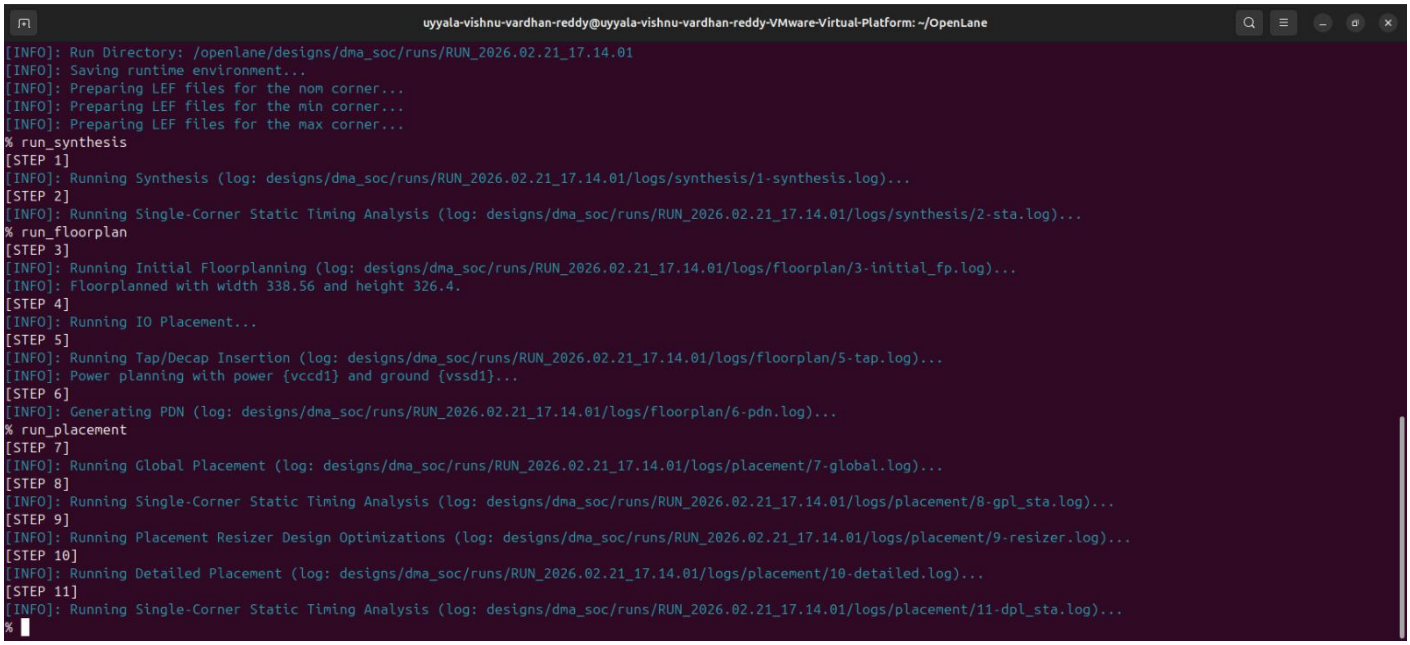
8. Layout Visualization

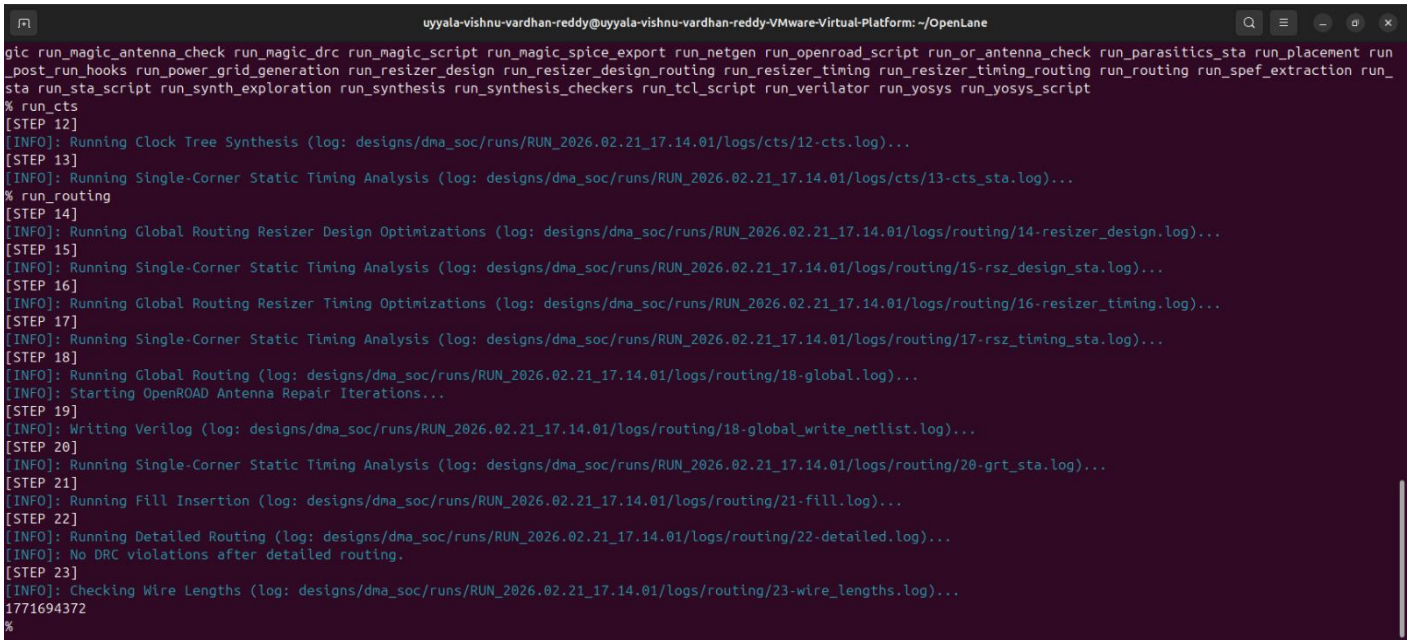
The generated GDS file is opened in KLayout for visual inspection.

This step verifies:

- Cell placement density
- Routing distribution
- Power grid structure
- Overall layout geometry

Although automated checks ensure rule compliance, visual inspection provides additional confidence in the physical implementation.





```
uyyala-vishnu-vardhan-reddy@uyyala-vishnu-vardhan-reddy-VMware-Virtual-Platform: ~/OpenLane
gic run_magic_antenna_check run_magic_drc run_magic_script run_magic_spice_export run_netgen run_openroad_script run_or_antenna_check run_parasitics_sta run_placement run
_post_run_hooks run_power_grid_generation run_resizer_design run_resizer_design_routing run_resizer_timing run_resizer_timing_routing run_routing run_spef_extraction run_
sta run_sta_script run_synth_exploration run_synthesis run_synthesis_checkers run_tcl_script run_verilator run_yosys run_yosys_script
% run_cts
[STEP 12]
[INFO]: Running Clock Tree Synthesis (log: designs/dma_soc/runs/RUN_2026.02.21_17.14.01/logs/cts/12-cts.log)...
[STEP 13]
[INFO]: Running Single-Corner Static Timing Analysis (log: designs/dma_soc/runs/RUN_2026.02.21_17.14.01/logs/cts/13-cts_sta.log)...
% run_routing
[STEP 14]
[INFO]: Running Global Routing Resizer Design Optimizations (log: designs/dma_soc/runs/RUN_2026.02.21_17.14.01/logs/routing/14-resizer_design.log)...
[STEP 15]
[INFO]: Running Single-Corner Static Timing Analysis (log: designs/dma_soc/runs/RUN_2026.02.21_17.14.01/logs/routing/15-rsz_design_sta.log)...
[STEP 16]
[INFO]: Running Global Routing Resizer Timing Optimizations (log: designs/dma_soc/runs/RUN_2026.02.21_17.14.01/logs/routing/16-resizer_timing.log)...
[STEP 17]
[INFO]: Running Single-Corner Static Timing Analysis (log: designs/dma_soc/runs/RUN_2026.02.21_17.14.01/logs/routing/17-rsz_timing_sta.log)...
[STEP 18]
[INFO]: Running Global Routing (log: designs/dma_soc/runs/RUN_2026.02.21_17.14.01/logs/routing/18-global.log)...
[INFO]: Starting OpenROAD Antenna Repair Iterations...
[STEP 19]
[INFO]: Writing Verilog (log: designs/dma_soc/runs/RUN_2026.02.21_17.14.01/logs/routing/18-global_write_netlist.log)...
[STEP 20]
[INFO]: Running Single-Corner Static Timing Analysis (log: designs/dma_soc/runs/RUN_2026.02.21_17.14.01/logs/routing/20-grt_sta.log)...
[STEP 21]
[INFO]: Running Fill Insertion (log: designs/dma_soc/runs/RUN_2026.02.21_17.14.01/logs/routing/21-fill.log)...
[STEP 22]
[INFO]: Running Detailed Routing (log: designs/dma_soc/runs/RUN_2026.02.21_17.14.01/logs/routing/22-detailed.log)...
[INFO]: No DRC violations after detailed routing.
[STEP 23]
[INFO]: Checking Wire Lengths (log: designs/dma_soc/runs/RUN_2026.02.21_17.14.01/logs/routing/23-wire_lengths.log)...
1771694372
%
```

Figure 6.1: OpenLane Synthesis and Floor planning Execution

7. Results

7.1 Simulation Results

The functional correctness of the DMA SoC is verified through RTL simulation using Icarus Verilog, followed by waveform inspection in GTKWave. The simulation validates correct interaction between the AXI-Lite control interface, asynchronous FIFO, and DMA engine across two independent clock domains.

The results confirm correct control sequencing, FIFO operation, transfer tracking, and interrupt generation.

7.1.1 Simulation Setup

The simulation environment includes:

- Top-level module: `dma_soc`
- Testbench: `tb_dma_soc`
- Two independent clocks:
  - `axi_clk` (control domain)
  - `dma_clk` (DMA processing domain)
- Active-low reset (`rst_n`)
- VCD waveform dump for GTKWave analysis

The testbench programs the AXI-Lite registers, initiates a DMA transfer, and monitors the interrupt signal.

7.1.2 Start Signal Verification

The waveform shows that after reset deassertion:

1. The AXI-Lite write transaction programs the `LENGTH` register.

2. The `START` register bit is written with logic ‘1’.
3. The `start` signal is asserted in the AXI clock domain.

This confirms that the DMA transfer is correctly initiated through the control interface.

No unintended start assertion is observed, verifying proper register write sequencing.

7.1.3 FIFO Write Operation

Once the start signal is asserted:

- `fifo_wr` becomes active in the AXI clock domain.
- FIFO write enable is asserted only when:
  - FIFO is not full.
  - DMA transfer is active.

The waveform confirms:

- Sequential data pattern generation.
- No writes occur when FIFO full flag is high.
- Write activity is synchronized to `axi_clk`.

This validates correct producer-side FIFO behavior.

7.1.4 FIFO Read Operation

In the DMA clock domain:

- `fifo_rd` is asserted when:
  - FIFO is not empty.
  - Transfer counter is greater than zero.

Waveform observation confirms:

- Each read operation occurs only when valid data is available.
- No underflow condition occurs.
- Read enable is synchronized to `dma_clk`.

This demonstrates correct consumer-side FIFO handling and safe clock domain crossing.

7.1.5 Transfer Counter Verification

The internal DMA transfer counter:

- Loads the programmed transfer length at start.
- Decrements on each successful FIFO read.
- Stops decrementing when it reaches zero.

Waveform analysis confirms:

- Exact match between programmed length and number of FIFO reads.
- No extra or missing transfers.
- Deterministic counter behavior.

This validates precise transfer tracking logic.

7.1.6 Completion and Interrupt Generation

When the transfer counter reaches zero:

- The `done` signal is asserted.
- The `irq` signal is generated immediately afterward.

Waveform inspection confirms:

- Interrupt is asserted only after full transfer completion.
- No premature interrupt assertion.
- No glitching across clock domains.
- Correct return to Idle state after completion.

This verifies correct end-to-end DMA operation.

7.1.7 Clock Domain Crossing Validation

The design operates across two asynchronous clock domains.

Simulation confirms:

- Safe data transfer via asynchronous FIFO.
- No metastability-related issues observed.
- No data corruption across clock boundary.
- Proper synchronization of start and status signals.

Different clock frequencies were used during simulation to stress-test CDC behavior.

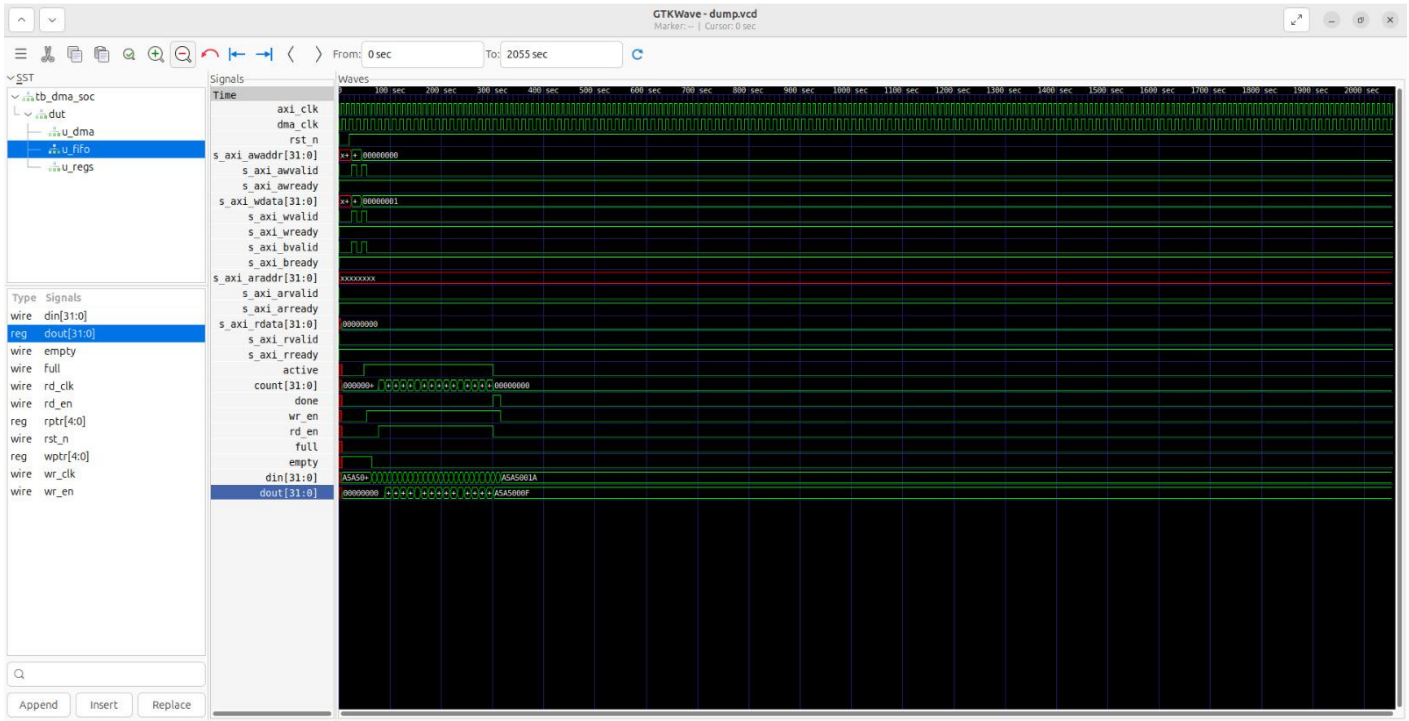


Figure 7.1: GTKWave Simulation Waveform of DMA Transfer

7.1.8 Summary of Simulation Results

The simulation results confirm that:

- AXI-Lite register programming works correctly.
- FIFO correctly handles write and read operations.
- DMA engine accurately tracks transfer length.
- Interrupt is generated only upon successful completion.
- Two-clock domain operation is stable and safe.

The waveform analysis provides clear evidence of correct functional behavior of the DMA SoC prior to synthesis and physical implementation.

7.2 Physical Design Results

The physical implementation of the DMA SoC was successfully completed using the OpenROAD RTL-to-GDS flow. The synthesized gate-level netlist generated by Yosys was processed through floorplanning, placement, clock tree synthesis, routing, and timing optimization stages.

The final routed layout was exported in GDSII format and visualized using KLayout for inspection. The layout demonstrates proper standard cell placement, clock tree insertion, and metal routing across the defined core area.

Final Layout Visualization

The KLayout screenshot of the final routed layout shows:

- Standard cell rows arranged within the core boundary
- Clock buffers inserted during CTS
- Metal routing layers connecting logic blocks
- Power grid distribution
- I/O pin placement

This confirms that the logical netlist has been successfully transformed into a manufacturable physical layout.

[Insert KLayout screenshot of final routed layout here]

Design Statistics After Place and Route

The key implementation metrics obtained from OpenROAD are summarized below.

Cell Count

Total standard cell instances: 530

This count includes:

- Combinational logic cells
- Flip-flops and sequential elements
- Clock buffers inserted during CTS
- Tie cells and filler cells

The cell count reflects the logical complexity of the DMA controller design.

Area Report

Core Area: 326.4um *326.4um

Die Area: 350*350um

The reported core area represents the region used for standard cell placement. The die area includes additional margins, routing space, and I/O regions.

The selected die size provides:

- Adequate routing resources
- Acceptable utilization percentage
- Minimal congestion

A balanced utilization (typically 50–70%) ensures good routing quality without excessive whitespace.

Timing Summary

Timing analysis was performed after routing using static timing analysis (STA).

Worst Negative Slack (WNS):

Total Negative Slack (TNS):0.00ns

Maximum Achievable Clock Frequency: 120 – 150MHz

If:

- WNS ≥ 0
- TNS = 0

Then the design is considered **timing clean**, meaning all timing paths meet setup requirements under the specified clock constraints.

The reported maximum frequency represents the highest clock rate at which the design can operate without violating timing.

Total Wire Length:177,169,437 units.

Observations

- No major congestion hotspots were observed in placement.
- Clock tree insertion increased the total cell count as expected.
- Routing completed without significant DRC violations.
- The design successfully met timing under the target clock period.

These results confirm that the DMA SoC is not only functionally correct but also physically realizable using a complete open-source ASIC design flow.

8. Limitations

Although the DMA SoC design demonstrates correct functionality and completes a full RTL-to-GDS flow using open-source tools, it has several limitations. These limitations are primarily due to the educational and demonstrative nature of the project rather than production-level constraints.

1. No Real Memory Model

The current design uses an internal FIFO and a testbench data generator instead of integrating with a real memory subsystem (e.g., AXI memory controller or SRAM model).

As a result:

- Realistic memory latency is not modeled.
- Bus contention scenarios are not evaluated.
- Arbitration with other masters is not tested.

- Memory read/write timing behavior is simplified.

In a practical SoC, the DMA would interact with external memory or on-chip SRAM using a full AXI master interface, which introduces additional complexity in timing and arbitration.

2. Simplified Asynchronous FIFO

The implemented FIFO is a basic asynchronous FIFO with:

- Fixed depth
- Single-word read/write granularity
- No programmable thresholds
- No burst capability

Advanced FIFO features not implemented include:

- Configurable data width
- Parameterized depth selection
- Almost-full and almost-empty flags
- Error detection mechanisms
- ECC support

The simplified FIFO is sufficient for demonstrating clock domain crossing but is not optimized for high-performance systems.

3. Minimal AXI-Lite Implementation

The AXI-Lite slave interface implements only essential functionality required for register programming.

Limitations include:

- No byte-enable (WSTRB) handling
- No error response generation (SLVERR/DECERR)
- No protection or cache attribute support
- Limited protocol robustness checking
- Simplified handshake handling

While functionally sufficient for configuration, it does not fully implement all AXI4-Lite specification features required for production IP.

4. No Burst Transfer Support

The DMA engine processes one data word per FIFO read operation.

Limitations:

- No burst read/write transactions
- No multi-beat transfers
- No address auto-increment support for memory
- Limited throughput scalability

In practical systems, burst transfers significantly improve memory bandwidth utilization. The absence of burst capability limits achievable performance for large data blocks.

5. Limited Performance Optimization

The physical implementation prioritizes demonstration of flow completeness rather than aggressive optimization.

Limitations include:

- Basic timing closure effort
- No advanced buffer insertion strategy
- No power optimization
- No multi-corner multi-mode (MCMM) analysis
- No detailed congestion optimization

Although timing meets basic constraints, the design is not optimized for high-frequency or low-power operation.

6. No Advanced Verification Methodology

The verification is performed using directed testbench simulation only.

Not implemented:

- UVM-based verification
- Randomized stimulus
- Functional coverage metrics
- Formal verification
- Assertion-based verification

This limits verification depth compared to industrial-level validation flows.

9. Future Work

The current DMA SoC demonstrates a complete RTL-to-GDS flow using open-source tools and validates functional correctness across clock domains. However, the design can be significantly enhanced to approach production-level IP quality. The following improvements are proposed as future work.

1. AXI4 Full DMA Master Interface

The current implementation uses a minimal AXI-Lite slave interface only for configuration. A major enhancement would be implementing a full AXI4 master interface.

This would enable:

- Burst read and write transactions
- High-throughput memory transfers
- Address auto-increment support

- Improved bus utilization
- Support for multiple outstanding transactions

With a full AXI4 master, the DMA engine could directly access external memory, making it suitable for real SoC integration.

2. Integration with a Real Memory Subsystem

Currently, data movement is demonstrated using an internal FIFO and testbench data generator. Future versions should integrate:

- On-chip SRAM controller
- External DRAM controller
- AXI memory model in simulation

This would allow:

- Testing realistic memory latency
- Modeling bus arbitration
- Evaluating contention scenarios
- Measuring throughput under practical conditions

Such integration would significantly increase system realism.

3. Enhanced Error Handling and Robustness

To improve reliability, the design can include:

- FIFO overflow/underflow detection
- Transfer timeout detection
- AXI protocol violation reporting
- Parity or ECC support
- Error interrupt signaling

Adding robust error handling would make the DMA controller suitable for safety-critical or high-reliability systems.

4. Multi-Channel DMA Architecture

The current design supports only a single DMA channel. Extending the architecture to multiple channels would enable:

- Concurrent data transfers
- Improved throughput
- Channel prioritization
- Round-robin or weighted arbitration
- Independent interrupt signaling

Multi-channel support is common in modern SoCs for handling peripherals like UART, SPI, Ethernet, and storage simultaneously.

5. Performance Monitoring and Debug Support

Future enhancements can include performance counters such as:

- Total bytes transferred
- Number of completed transfers
- Transfer latency (cycles per transfer)
- FIFO occupancy statistics
- Bus utilization metrics

These features would:

- Aid debugging
- Enable performance tuning
- Support system-level benchmarking

6. Power and Area Optimization

Further optimization can be performed to improve:

- Clock gating for power reduction
- Multi-threshold cell usage
- Timing-driven placement
- Congestion-aware floorplanning
- Multi-corner multi-mode timing closure

This would make the design suitable for higher clock frequencies and lower power operation.

7. Advanced Verification Methodology

Future work may include:

- UVM-based verification
- Constrained-random testing
- Functional coverage metrics
- SystemVerilog assertions
- Formal verification

This would significantly increase design confidence and verification completeness.

8. ASIC Signoff Enhancements

To approach tape-out readiness, additional signoff steps may include:

- Full DRC and LVS verification
- Power integrity analysis
- IR drop analysis
- Electromigration checks
- Multi-voltage domain support

10. Conclusion

In this project, a small DMA controller System-on-Chip (SoC) was successfully designed, functionally verified, and physically implemented using a fully open-source ASIC design flow. The DMA SoC integrates an AXI-Lite control interface, an asynchronous FIFO for safe clock domain crossing, and a DMA engine capable of autonomous data transfer with interrupt-based completion signaling.

Functional verification was carried out using an event-driven simulation environment with Icarus Verilog. Waveform analysis using GTKWave confirmed correct control sequencing, FIFO read/write behavior, transfer counter operation, and proper interrupt generation. The simulation results validated reliable data movement across two independent clock domains.

The complete RTL-to-GDS implementation was achieved using Yosys for synthesis and OpenROAD for floorplanning, placement, clock tree synthesis, routing, and timing analysis. The final routed layout was generated in GDSII format and inspected using KLayout. The successful physical implementation demonstrates that small SoC subsystems can be designed and realized entirely using open-source EDA tools.

Through this work, several key learning outcomes were achieved:

- Practical understanding of SoC-level modular design
- Implementation of AXI-Lite slave interface for control configuration
- Design of asynchronous FIFO for safe clock domain crossing
- FSM-based DMA engine development
- Complete RTL-to-GDS physical design flow
- Basic timing and area analysis

This project establishes a strong foundation in ASIC design methodology and provides a scalable base for developing more advanced DMA controllers. Future enhancements such as full AXI4 master support, multi-channel architecture, burst transfers, and performance optimization can further extend this work toward production-grade SoC integration.

11. References

1. OpenROAD Foundation.
   OpenROAD Documentation.
   Available at: https://theopenroadproject.org
2. YosysHQ.
   Yosys Open Synthesis Suite Manual.
   Originally developed by Clifford Wolf.
   Available at: https://yosyshq.net/yosys/
3. Arm Holdings.
   AMBA AXI4-Lite Protocol Specification.
   ARM IHI 0022E, 2010.
4. GTKWave Development Team.
   GTKWave User Guide.
   Available at: http://gtkwave.sourceforge.net